

Python for Computer Science and Data Science 2 (CSE 3652)

MINOR ASSIGNMENT-4: MACHINE LEARNING- CLASSIFICATION, REGRESSION AND CLUSTERING

Q1. Perform dimensionality reduction using scikit-learn's TSNE estimator on the Iris dataset, then graph the results.

Code:

```
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import pandas as pd

iris = load_iris()
X, y = iris.data, iris.target

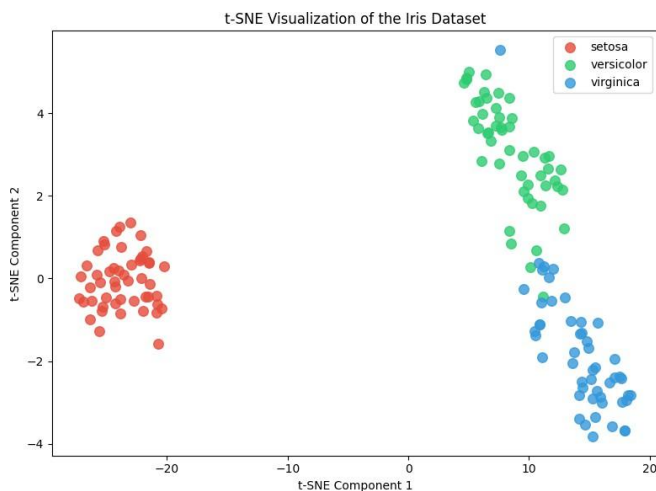
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X)

df_tsne = pd.DataFrame(X_tsne, columns=['TSNE-1', 'TSNE-2'])
df_tsne['target'] = y

colors = ['#e74c3c', '#2ecc71', '#3498db']
plt.figure(figsize=(8, 6))
for label, name in enumerate(iris.target_names):
    subset = df_tsne[df_tsne['target'] == label]
    plt.scatter(subset['TSNE-1'], subset['TSNE-2'], label=name, s=70, color=colors[label],
alpha=0.8)

plt.title('t-SNE Visualization of the Iris Dataset')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.legend()
plt.tight_layout()
plt.show()
```

Output:



Name: Swayam Swarup Jethi | Reg No: 2341010085

Q2. Create a Seaborn pairplot graph for the California Housing dataset. Use Matplotlib features for panning and zooming.

Code:

```
from sklearn.datasets import fetch_california_housing
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

```

housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)

df['MedHouseVal'] = housing.target

df_sample = df.sample(n=300, random_state=42)

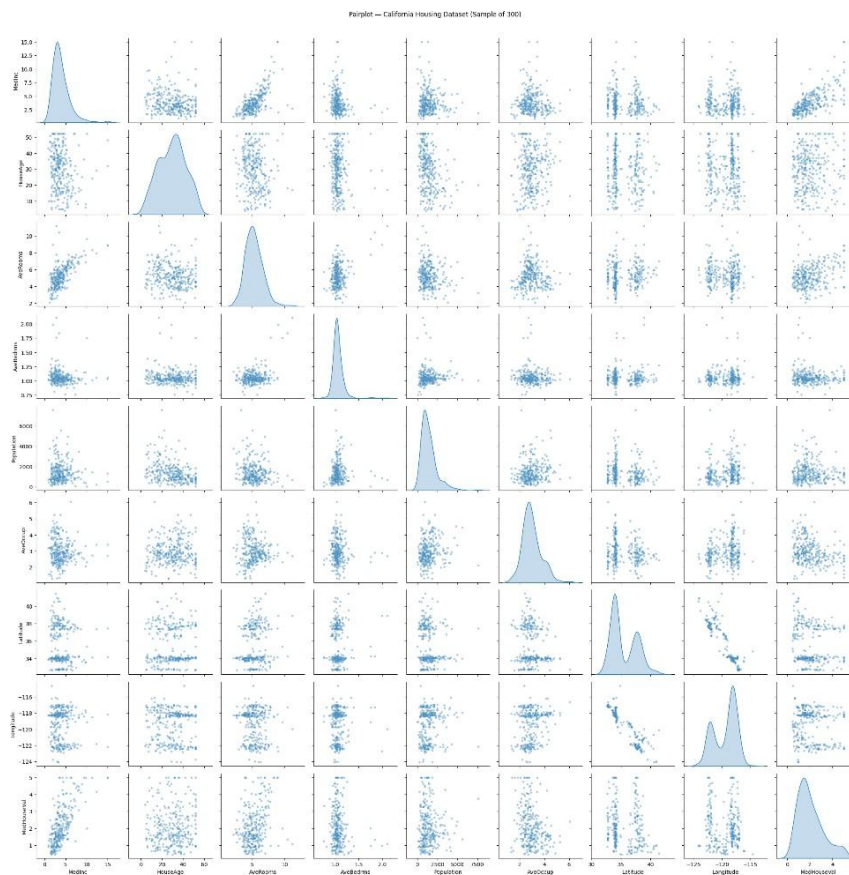
sns.pairplot(
    df_sample,
    diag_kind='kde',
    plot_kws={'alpha': 0.5, 's': 20}
)

plt.suptitle('Pairplot - California Housing Dataset',y=1.02)

plt.show()

```

Output:



Name: Swayam Swarup Jethi | Reg No: 2341010085

Q3. Download NYC annual and January temperature data from NOAA (1895–2025). Implement simple linear regression and compare trends.

Code:

```

import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Load CSV file
df = pd.read_csv("temperature.csv")

# X = Year, Y = Temperature
X = df[['Year']]
y = df['Temperature']

# Train model
model = LinearRegression()

```

```

model.fit(X, y)

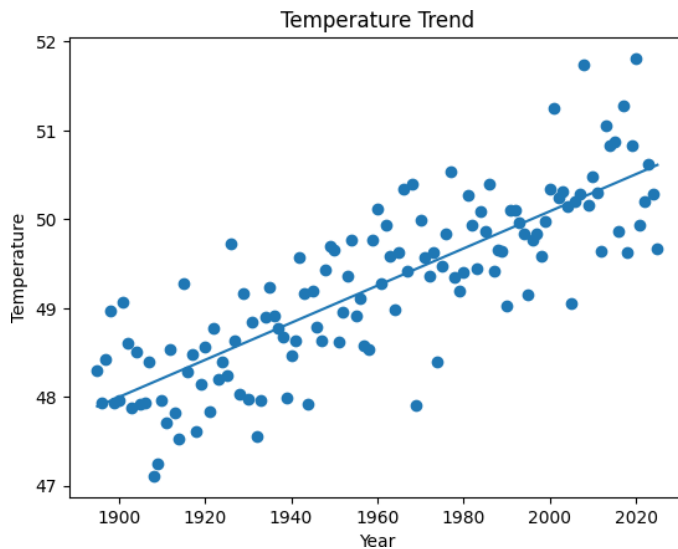
y_pred = model.predict(X)

plt.scatter(X, y)
plt.plot(X, y_pred)
plt.xlabel("Year")
plt.ylabel("Temperature")
plt.title("Temperature Trend")
plt.show()

print("Slope:", model.coef_[0])

```

Output:



Slope: 0.02090231142902899

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q4. Load the Iris dataset and perform classification using KNeighborsClassifier (default k). What is the prediction accuracy?

Code:

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = KNeighborsClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

```

Output:

Accuracy: 1.0

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q5. Classify a new point $P=(3.0, 3.0)$ using KNN ($k=3$) on the given 2D dataset with Euclidean distance.

Dataset: A(2,3)→Class 0, B(1,1)→Class 0, C(4,4)→Class 1, D(5,2)→Class 1

Code:

```
import math

# Dataset
points = {
    'A': (2, 3, 0),
    'B': (1, 1, 0),
    'C': (4, 4, 1),
    'D': (5, 2, 1)
}

# New point
P = (3, 3)

# Calculate distance
for name, (x, y, cls) in points.items():
    dist = math.sqrt((x - P[0])**2 + (y - P[1])**2)
    print(name, "Distance =", round(dist, 2), "Class =", cls)
```

Output:

```
A Distance = 1.0 Class = 0
B Distance = 2.83 Class = 0
C Distance = 1.41 Class = 1
D Distance = 2.24 Class = 1
```

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q6. Classify a new student [72, 78, 75] using KNN ($k=3$) on the given exam score dataset (Pass/Fail).

Code:

```
import math

# Dataset
students = [
    [85, 90, 88, "Pass"],
    [70, 60, 80, "Pass"],
    [50, 45, 70, "Fail"],
    [95, 75, 92, "Pass"],
    [65, 55, 50, "Fail"],
    [58, 48, 96, "Fail"]
]

# New student
new_student = [72, 78, 75]

distances = []

# Calculate distances
for s in students:
    dist = math.sqrt(
        (s[0]-new_student[0])**2 +
        (s[1]-new_student[1])**2 +
        (s[2]-new_student[2])**2
    )

    distances.append((dist, s[3]))

# Sort distances
distances.sort()

# Take k=3 nearest neighbors
k = 3
neighbors = distances[:k]

print("Nearest Neighbors:")
print(neighbors)

pass_count = 0
```

```

fail_count = 0

for n in neighbors:
    if n[1] == "Pass":
        pass_count += 1
    else:
        fail_count += 1

# Prediction
if pass_count > fail_count:
    print("Prediction: Pass")
else:
    print("Prediction: Fail")

```

Output:

```

Nearest Neighbors:
[(18.788294228055936, 'Pass'), (21.95449840010015, 'Pass'), (28.75760768909681, 'Pass')]
Prediction: Pass

```

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q7. Using scikit-learn's KFold and cross_val_score, determine the optimal k for KNeighborsClassifier on the Iris dataset.

Code:

```

from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)

best_k = 1
best_score = 0

# Try different k values
for k in range(1, 11):
    model = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(model, X, y, cv=kf)

    avg_score = scores.mean()
    print("k =", k, "Accuracy =", avg_score)
    if avg_score > best_score:
        best_score = avg_score
        best_k = k

print("\nBest k =", best_k)

```

Output:

```

k = 1 Accuracy = 0.96
k = 2 Accuracy = 0.96
k = 3 Accuracy = 0.9666666666666668
k = 4 Accuracy = 0.9733333333333334
k = 5 Accuracy = 0.9733333333333334
k = 6 Accuracy = 0.9733333333333334
k = 7 Accuracy = 0.9733333333333334
k = 8 Accuracy = 0.9733333333333334
k = 9 Accuracy = 0.9666666666666668
k = 10 Accuracy = 0.9733333333333334

```

```

Best k = 4

```

Name: Swayam Swarup Jethi | Reg No: 2341010085

Q8. Perform K-Means clustering (k=2) on the dataset {(1,1),(2,2),(3,3),(8,8),(9,9),(10,10)} and visualize the clusters.

Code:

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

X = [[1,1],[2,2],[3,3],[8,8],[9,9],[10,10]]

kmeans = KMeans(n_clusters=2, random_state=42)

kmeans.fit(X)

labels = kmeans.labels_

for i in range(len(X)):
    plt.scatter(X[i][0], X[i][1], c=f"C{labels[i]}")

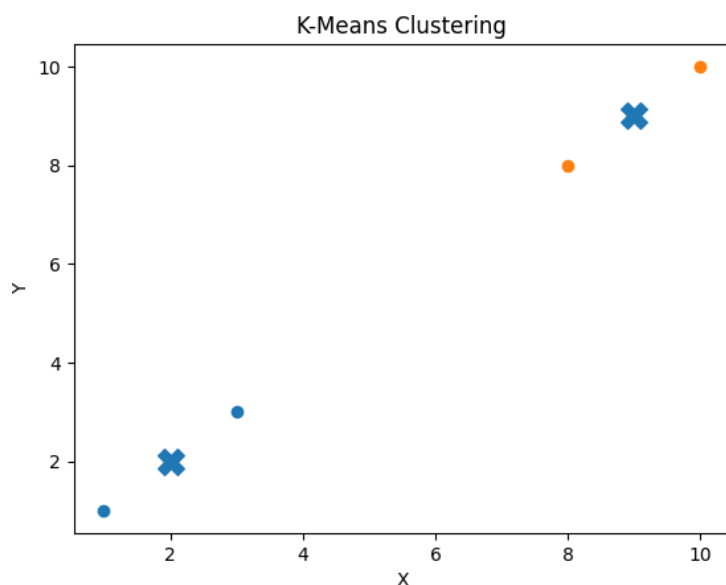
centers = kmeans.cluster_centers_

plt.scatter(centers[:,0],centers[:,1],marker='x',s=200)

plt.title("K-Means Clustering")
plt.xlabel("X")
plt.ylabel("Y")

plt.show()
```

Output:



Name: Swayam Swarup Jethi | Reg No: 2341010085

Q9. Perform K-Means clustering on the Mall Customer Segmentation dataset (k=5). Use the Elbow Method and visualize clusters.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

df = pd.read_csv("Mall_Customers.csv")
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
wcss = []

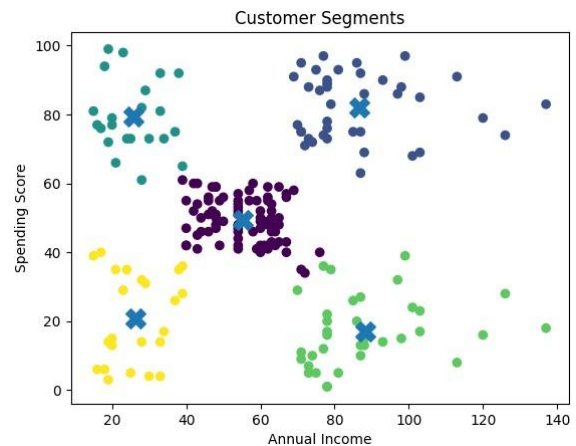
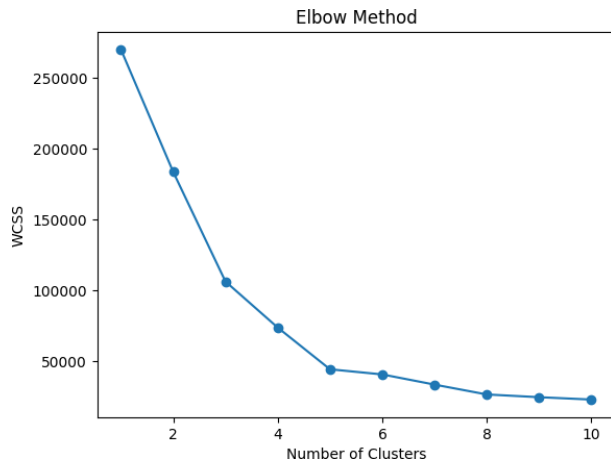
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,11), wcss, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
```

```
plt.show()
kmeans = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans.fit_predict(X)
plt.scatter(X.iloc[:,0],X.iloc[:,1],c=y_kmeans)
plt.scatter(kmeans.cluster_centers_[:,0],kmeans.cluster_centers_[:,1],s=200,marker='X')
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Segments")

plt.show()
```

Output:



Name: Swayam Swarup Jethi | Reg No: 2341010085

Q10. Perform the following tasks using the pandas Series object: (a) from list, (b) all 100.0, (c) 20 random numbers with describe(), (d) temperatures with custom index, (e) Series from dictionary.

Code:

```
import pandas as pd
import numpy as np

# (a) Create Series from list
s1 = pd.Series([7, 11, 13, 17])
print("(a) Series from list:")
print(s1)

# (b) Series with five elements of 100.0
s2 = pd.Series([100.0] * 5)
print("\n(b) Series with 100.0:")
print(s2)

# (c) Random Series and statistics
s3 = pd.Series(np.random.randint(0, 101, 20))
print("\n(c) Random Series:")
print(s3)

print("\nDescriptive Statistics:")
print(s3.describe())

# (d) Temperature Series with custom index
temperatures = pd.Series(
    [98.6, 98.9, 100.2, 97.9],
    index=['Julie', 'Charlie', 'Sam', 'Andrea']
)

print("\n(d) Temperature Series:")
print(temperatures)

# (e) Create Series from dictionary
```

```
temp_dict = {
    'Julie': 98.6,
    'Charlie': 98.9,
    'Sam': 100.2,
    'Andrea': 97.9
}

s5 = pd.Series(temp_dict)

print("\n(e) Series from Dictionary:")
print(s5)
```

Output:

(a) Series from list:

```
0      7
1     11
2     13
3     17
dtype: int64
```

(b) Series with 100.0:

```
0     100.0
1     100.0
2     100.0
3     100.0
4     100.0
dtype: float64
```

(c) Random Series:

```
0      6
1     43
2     57
3     21
4     85
5     36
6     63
7     83
8      4
9     54
10     34
11     76
12     79
13     39
14     62
15     85
16      1
17     51
18     45
19     28
dtype: int64
```

Descriptive Statistics:

```
count    20.000000
mean     47.600000
std      26.896292
min       1.000000
25%      32.500000
50%      48.000000
75%      66.250000
max      85.000000
dtype: float64
```

(d) Temperature Series:

```
Julie      98.6
Charlie    98.9
Sam       100.2
Andrea     97.9
dtype: float64
```

(e) Series from Dictionary:

```
Julie      98.6
Charlie    98.9
Sam       100.2
Andrea     97.9
dtype: float64
```